

Gerador de Cortes — Clipper

Relatório técnico do projeto + auditoria pré-execução
foco: vídeo de ~17 GB / 1h40 em máquina 16 GB RAM + Intel Arc B580 12 GB

Data: 2026-09-18 **Estado do código:** pós-correções da auditoria (commits fix/*) **Veredito:** SAFE TO BEGIN VIDEO TEST (começando pelo menor vídeo real)

Regra da auditoria: nenhum vídeo real foi executado. Análise estática do código + testes controlados com vídeos sintéticos gerados por ffmpeg (testsrc/sine/voz espeak-ng, /tmp, poucos MB).

Sumário

1. Visão geral
2. Arquitetura
3. Fluxo completo
4. GPU — Intel Arc B580
5. CPU — Ryzen 5 4600G
6. Memória (RAM/VRAM/buffers/temporários)
7. FFmpeg
8. NVIDIA API (scoring)
9. Métricas por execução
10. Segurança e estabilidade
11. Estrutura do projeto
12. Configuração
13. Instalação
14. Execução
15. Diagnóstico
16. Auditoria (achados e correções)
17. Limitações e conclusão

1. Visão geral

O que é: programa Python que recebe a gravação de uma live/gameplay (mp4/mkv, tipicamente ~17 GB / 1h40), transcreve o áudio localmente, usa um LLM (tier grátis da NVIDIA Build) para pontuar os melhores momentos e entrega clipes verticais 9:16 com legenda queimada, prontos para TikTok/Reels/Shorts.

Problema que resolve: assistir horas de live para achar cortes manualmente. Fluxo pensado em ritmo diário: sobe o vídeo de manhã, clipes prontos ao longo do dia, custo zero (Whisper local + free tier NVIDIA).

Objetivo desta auditoria: provar que o pipeline atual é seguro para o vídeo grande: sem RAM/VRAM descontrolada, sem disco cheio, sem processos órfãos, sem perda do original, com fallback explícito e relatório por execução.

```
vídeo 17 GB/1h40 ■■ transcrição local ■■ scoring LLM ■■ seleção
■■ corte+legenda (ffmpeg) ■■ 8 clipes 9:16 + manifest.json
■■ metrics/<timestamp>_<video>_<id>.json
```

2. Arquitetura

Orquestrador fino (**clipper.py**, ~200 linhas: só CLI + ordem das etapas) sobre módulos **core/*** com responsabilidade única (SRP). Comunicação por objetos de domínio (**core/models.py**: Word/Segment/Candidate) passados em sequência — sem fila, sem concorrência entre etapas, um vídeo por processo.

```
clipper.py (CLI/orquestrador)
■■■ core/preflight.py  validação fail-fast (env, ffmpeg, disco, GPU)
■■■ core/transcribe.py orquestra backends (vulkan→openvino→cpu)
■      ■■■ core/backends.py whisper.cpp / OpenVINO em chunks de 30 s
■■■ core/candidates.py janelas 20–90 s + snap em fronteira de palavra
■■■ core/audio.py      energia via ffmpeg volumedetect (1 proc/candidato)
■■■ core/scoring.py    LLM NVIDIA em lotes de ~6 (+retry limitado)
■■■ core/selection.py  NMS com decaimento + diversidade por 10 min
■■■ core/video.py      OpenCV (3 frames/clipe) + ffmpeg por clipe
■■■ core/cache.py      transcrição/scores por fingerprint tamanho+mtime
■■■ core/metrics.py    1 relatório JSON por execução + resumo terminal
```

Dependências externas: ffmpeg/ffprobe (obrigatórios), faster-whisper/CTranslate2 (CPU), openai SDK (scoring), psutil (métricas), opcional: openvino+optimum-intel+transformers (GPU), binário whisper.cpp (Vulkan), opencv (rosto). Nenhum serviço local, nenhum daemon.

Fluxo de dados: caminho do vídeo (string) → segmentos → candidatos → candidatos pontuados → selecionados → arquivos .mp4 + manifest.json + metrics/*.json. A transcrição completa nunca sai da máquina; à NVIDIA vão só trechos de texto (até 1500 chars por candidato).

3. Fluxo completo

3.1 Entrada e validação (preflight)

Checa: arquivo existe e tem trilha de áudio; ffmpeg presente; pasta de saída gravável; **espaço livre ≥ 1 GB (falha) / ≥ 5 GB (aviso)**; NVIDIA_API_KEY definida; imprime specs + limites + relatório GPU/backend (nunca silencioso). Falhas abortam antes de qualquer custo (exit 1) e geram relatório *failed/preflight*.

3.2 Extração e transcrição

Backend **auto**: whisper.cpp/Vulkan → OpenVINO/GPU → faster-whisper/CPU, cada um tentado 1× com motivo impresso no fallback. Backends GPU extraem áudio em **chunks de 30 s (~1 MB cada, 1 por vez, removidos após uso)** via ffmpeg — o vídeo de 17 GB nunca é carregado. O caminho CPU decodifica o áudio inteiro uma vez (~0,4 GB para 100 min — aceitável, ver §6). Modelo int8, 4 threads, saída em segmentos com palavras.

3.3 Janelas, snap, áudio

Janela deslizante 20–90 s passo 20 s (~300 candidatos p/ 100 min). Snap ajusta início/fim à palavra mais próxima (±1,5 s) + respiro --pad. Energia (alta/media/baixa) medida com 1 ffmpeg *volumedetect* por candidato (timeout 8 s) + *speech_rate* (palavras/s).

3.4 Scoring (LLM)

Lotes de ~6 candidatos → 1 chamada de chat por lote (~50 chamadas p/ 100 min). Prompt com critérios virais + duração/energia/speech_rate + contexto/few-shot opcionais. Retry 3× backoff 10/30/60 s; lote que falha marca candidatos como *failed* (nunca nota 0 silenciosa). Resultado: score 0–10, título ≤50 chars, 3 hashtags, motivo.

3.5 Seleção

Filtra min-score, ordena, aplica NMS com decaimento (penaliza overlap em vez de descartar) e limite por janela de 10 min (diversidade). Sem I/O, O(n²) trivial para n≈300.

3.6 Detecção + corte + encode

Por clipe: OpenCV Haar lê **3 frames** para centralizar o crop no rosto (fallback: centro); SRT construído das palavras (pausa >0,4 s, 32 chars/linha, mín. 1 s); 1 ffmpeg por clipe com seek duplo (coarse+fine), filtros scale/crop/pad/subtitles, encode **h264_qsv** (B580) com fallback **libx264**, timeout 600 s. Falha num clipe não aborta os demais.

3.7 Saída e métricas

Clipes + manifest.json na pasta --out; cache opcional; relatório JSON próprio em metrics/ + resumo EXECUTION METRICS no terminal — em sucesso, falha ou Ctrl+C (*interrupted*).

4. GPU — Intel Arc B580 12 GB

Por que não faster-whisper na GPU: CTranslate2 é CUDA-only e nunca usa Intel GPU — por isso existem backends próprios. **Detecção** (core/intel.py): vendor 8086 em /sys/class/drm ou lspci; nome amigável do lspci; devices do OpenVINO; binário whisper.cpp no PATH; **sonda funcional QSV** (encode real de 10 frames, timeout 15 s, cacheada 1×/processo).

Uso: transcrição (whisper.cpp/Vulkan ou Whisper via optimum-intel device GPU, 1 instância, chunks de 30 s) e encode h264_qsv. Filtros e Haar ficam na CPU por serem leves. **Nesta máquina (verificado):** B580 presente, mas OpenVINO enxerga só CPU (falta driver Level Zero/OpenCL) e whisper.cpp ausente → backend efetivo CPU com motivo explícito; QSV funcional → encode na GPU.

Fallback (nunca silencioso): qualquer backend GPU levanta RuntimeError(motivo) 1×; o orquestrador imprime *GPU transcription backend unavailable: <motivo> / Falling back to CPU*. OOM em GPU também cai nesse caminho (falha controlada, sem loop de realocação). **Margem VRAM:** whisper-medium fp16 ≈ 2–4 GB de 12 GB; nada tenta usar tudo.

Para ativar a B580 na transcrição: instalar intel-opencl-icd + level-zero (Fedora) e/ou whisper.cpp com Vulkan + modelo ggml-medium.

5. CPU — Ryzen 5 4600G (6C/12T)

Princípio: **B580 = pesado, 4600G = coordenação + auxiliar**. Ficam na CPU por motivo: faster-whisper (fallback, int8, 4 threads), decode de áudio/ffmpeg auxiliares, filtros de vídeo e Haar (leves), orquestração e I/O, sampler de métricas. Nada cria pool sem limite: OMP/MKL/OpenBLAS=4, faster-whisper cpu_threads=4 num_workers=1, ffmpeg=2 threads.

Etapas	Threads/processos	Porquê é seguro
Transcrição CPU	4 threads, 1 worker	int8, limite explícito; medido RTF ~0,6–0,9
ffmpeg cortes	2 threads, 1 proc/clipe sequencial	timeout 600 s; QSV tira o encode da CPU
Áudio (volumedetect)	1 proc/candidato sequencial	timeout 8 s; ~0,5–1 s cada
OpenCV rosto	3 frames/clipe, sequencial	milissegundos; sem batch
Métricas	1 thread daemon, 1 amostra/2 s	só agregados escalares

6. Memória (RAM / VRAM / buffers / temporários)

RAM (14 GB úteis nesta máquina): nada carrega o vídeo. Picos medidos em teste sintético: ~0,9 GB (small/CPU). Estimativa p/ 1h40: modelo medium int8 ~1 GB + decode de áudio ~0,4 GB + segmentos/candidatos (MBs) + ffmpeg por clipe (centenas de MB transitórios) → poucos GB, com folga. Cache de transcrição em disco (JSON de ~MBs), nunca em RAM além do necessário.

VRAM (12 GB): 1 instância de modelo por vez; whisper-medium OV $\approx$ 2–4 GB; QSV usa VRAM só durante cada encode; sem batches, sem acúmulo entre chunks (cada chunk libera antes do próximo).

Temporários: chunks WAV 30 s (~1 MB, 1 vivo por vez, dir em \$TMPDIR removido inclusive em erro via finally); SRT por clipe (KBs, unlink em finally); sem cópia do vídeo em nenhum ponto.

Estratégia p/ vídeos grandes: streaming/chunks + geradores + 1 subprocesso por vez + timeouts em tudo + falha antes de disco cheio. Crescimento em memória é O(segmentos) — linear e pequeno (texto).

7. FFmpeg

Usado em: extração de chunks (pcm_s16le 16 kHz mono), volumedetect por candidato, sondas fprobe, corte+encode por clipe. **Decode sempre software;** encode h264_qsv (sonda funcional) com fallback libx264. Todos via `subprocess.run` com timeout, sem shell=True, sem pipes ilimitados (stderr capturado é de KBs; stdout DEVNULL ou JSON pequeno).

Subprocesso	Timeout	Saída capturada	Órfãos?
ffprobe (dimensões/duração)	10–15 s	JSON pequeno	run() aguarda; kill em timeout
chunk de áudio (1 por vez)	120 s	arquivo ~1 MB	mesmo acima; tmp removido
volumedetect	8 s	texto curto	mesmo acima
whisper.cpp por chunk	300 s	JSON do chunk	mesmo acima
corte+encode por clipe	600 s	stderr (KBs)	mesmo acima; Ctrl+C de terminal mata o grupo
sonda QSV (1×/processo)	15 s	null	mesmo acima

8. NVIDIA API (scoring)

O vídeo NÃO é enviado à NVIDIA API. Nem o áudio. Vai apenas texto: por lote, ~6 candidatos com transcript truncado em 1500 chars + tempos/duração/energia/speech_rate, mais prompt de sistema e few-shot. ~50 chamadas p/ 100 min (~300 candidatos). Resposta limitada (max_tokens=4000).

Robustez (pós-auditoria): client com `timeout=180 s` e `max_retries=0` (antes: default 600 s + retries internos do SDK); única política de retry é `_call_with_retry` (3×, backoff 10/30/60 s, limitado). HTTP/JSON inválido/conexão → lote marcado failed, pipeline continua. Métricas registram requests/sucessos/falhas/retries/latências e tokens *somente se a API retornar*; custo sempre null (sem tabela de preços).

Operacional: o modelo padrão meta/llama-3.3-70b-instruct expirou (410 Gone em 2026-08-26); usar --model com slug vivo, ex. nvidia/nemotron-3-super-120b-a12b.

9. Métricas por execução

Cada vídeo gera **seu próprio relatório** (nada é sobrescrito; timestamp+id evitam colisão; falhas e Ctrl+C também geram): resumo EXECUTION METRICS no terminal + JSON em *metrics/* com vídeo, execução (tempos por etapa, args, erro), transcrição (backend, tempo, RTF, segmentos), GPU/CPU/RAM (média/pico), FFmpeg e API NVIDIA. Sem transcrição, sem vídeo, sem arquivos grandes no JSON (poucos KB).

Leveza (auditada): 1 thread daemon, 1 amostra/2 s, só escalares (média/pico — jamais séries); ffprobe com timeout; nvidia-smi só se o binário existir (aqui: ausente → null honesto). Comparar execuções: mesmo schema_version, mesmos campos — RTF, tempo/clipe, RAM peak, backend e args lado a lado.

10. Segurança e estabilidade

Proteção	Como	Onde
Threads limitados	CPU 4 / ffmpeg 2 / OMP-MKL-OpenBLAS 4	core/config.py, transcribe.py
Sem teto virtual falso	RLIMIT_AS removido (limitava VIRT, não RSS; herdado pelo ffmpeg)	core/transcribe.py
Timeouts em tudo	2 s–600 s por tipo de subprocesso; API 180 s	todos os core/*.py
Retries limitados	API 3×+backoff; GPU 1× cada; QSV→x264 1×	scoring/transcribe/video.py
Disco	falha < 1 GB livre; aviso < 5 GB	core/preflight.py
Original intacto	guarda saída==entrada; temporários separados; -y só em saídas	clipper.py, core/video.py
Ctrl+C	relatório interrupted + propagação; filhos morrem com o grupo	clipper.py
Temporários	finally/GC: chunks, SRT; sem cópia do vídeo	core/backends.py, video.py

11. Estrutura do projeto

Arquivo	Responsabilidade → quem usa → por que existe
clipper.py	CLI + ordem das etapas + relatório/falha por execução → ponto de entrada → orquestrador fino, sem lógica de negócio
core/models.py	Word/Segment/Candidate (+overlap) → todos os módulos → contratos de dados sem dependências
core/config.py	Constantes + env + limites auto-detectados → todo o projeto → única fonte de verdade
core/system.py	Detecta CPU/RAM e calcula threads/limites → config+preflight → não travar PC fraco
core/intel.py	Detecta B580, resolve backend, sonda QSV (cacheada) → preflight/transcribe/video → GPU explícita, sem side-effect
core/preflight.py	Validação fail-fast (arquivo, ffmpeg, disco, API) → clipper → falhar antes de gastar horas
core/transcribe.py	Orquestra vulkan→openvino→cpu com fallback explícito → clipper → GPU primeiro, CPU seguro
core/backends.py	Chunks 30 s + inferência GPU 1× cada → transcribe → 17 GB sem carregar nada
core/candidates.py	Janelas 20–90 s + snap em palavra → clipper → cortes em frase inteira
core/audio.py	Energia por janela (volumedetect) → scoring → pista de hype/grito sem librosa
core/scoring.py	Prompt + LLM em lotes + retry limitado → clipper → julgamento viral
core/selection.py	NMS com decaimento + diversidade → clipper → top-N espalhado
core/video.py	Rosto (3 frames) + SRT + ffmpeg/QSV → clipper → mp4 9:16 legendado
core/cache.py	Fingerprint tamanho+mtime + JSONs → transcribe/scoring → não retranscrever à toa
core/metrics.py	Sampler leve + JSON por execução + resumo → clipper → benchmark/comparação

12. Configuração

Variável / Flag	Padrão	Efeito
NIM_MODEL / --model	meta/llama-3.3-70b-instruct (EOL!)	modelo de scoring (ver §8)
NVIDIA_API_KEY	(obrigatória)	auth do scoring; sem ela o preflight falha
CLIPPER_TRANSCRIBE_BACKEND / --transcribe-backend	auto	auto vulkan openvino cpu
WHISPER_MODEL_SIZE	medium (small se RAM ≤6 GB)	tamanho do Whisper
CLIPPER_CPU_THREADS	4 (auto p/ 4600G)	threads do Whisper/OpenMP
CLIPPER_FFMPEG_THREADS	2 (auto)	threads por ffmpeg
--top / --min-score / --max-per-10min	8 / 6.0 / 2	seleção de clipes
--pad / --no-vertical / --no-captions / --no-audio-features	0.8 s / off / off / off	corte e filtros
--cache-dir / --force-* / --context / --examples	—	cache, prompt, few-shot

Diretórios: *cortes/* (saídas), *metrics/* (relatórios, 1/execução), *.cache/* opcional (transcrição+scores por fingerprint). Nenhum arquivo de config — só env + CLI (YAGNI).

13. Instalação

Camada	Pacotes
Python (pip install -r requirements.txt)	faster-whisper, openai, opencv-python; opcional p/ B580: openvino, optimum-intel, transformers, soundfile
Sistema (Fedora)	ffmpeg (ffmpeg E ffprobe), python3; voz sintética só p/ testes: espeak-ng
GPU/driver (p/ transcrever na B580)	intel-onecl-icd, level-zero, intel-media-driver; verificar: vainfo + devices do OpenVINO
FFmpeg	com libx264 + h264_qsv; QSV verificado por sonda funcional, não por flag

14. Execução

```
export NVIDIA_API_KEY="nvapi-..."
python clipper.py video-1h40.mkv --out cortes/ --top 8
# saída: cortes/*.mp4 + manifest.json + metrics/<ts>_<video>_<id>.json
python clipper.py live.mp4 --cache-dir .cache/clipper # reuso sem recusto
```

Recomendação pós-auditoria: rodar primeiro o menor vídeo real e conferir o JSON (RTF, RAM peak, backend) antes dos maiores.

15. Diagnóstico

Sintoma	Causa provável	Como confirmar
Transcription backend: CPU (...)	Sem Level Zero/OpenCL ou sem whisper.cpp	ov devices; vainfo; which whisper-cpp
GPU transcription backend unavailable	Motivo impresso na linha (driver/modelo/binário)	ler o motivo; fallback é explícito
Erro NIM 410 Gone	Modelo expirado (padrão EOL)	listar /v1/models; trocar --model
Rate limit / JSON inválido	Free tier instável	retries no log; lote marcado failed; metrics.nvidia_api
ffmpeg falhou num clipe	Filtros/crop ou QSV	! QSV falhou → fallback libx264 é automático
Nenhum candidato / sem fala	Vídeo sem fala ou VAD removeu tudo	segments=0 no JSON
Travamento/OOM	Fora do esperado pós-auditoria	RAM peak no JSON; dmesg; free
Disco	Preflight falha < 1 GB / avisa < 5 GB	df -h na pasta --out

16. Auditoria (achados e correções)

Método: leitura integral do código; grep por shell=True/os.system/rm/while True/bare-except (zero ocorrências); inspeção de todos os subprocess.run (todos com timeout exceto a chamada de API — corrigida); verificação de defaults do SDK OpenAI; testes controlados só com vídeos sintéticos em /tmp (testsrc/sine/espeak-ng) — **nenhum vídeo real executado**. Contagem final:

```
CRITICAL: 0
HIGH: 4 (todos corrigidos)
MEDIUM: 6 (1 com hardening aplicado, 5 documentados)
LOW: 3 (documentados)
```

HIGH — corrigidos

ID	Achado (arquivo)	Risco / Impacto	Solução	Status
H1	Teto de memória virtual RLIMIT_AS 6 GB (core/transcribe.py)	Limitava VIRT (mmaps/stacks), não RSS; causaria MemoryError aleatórios em runs longos e era herdado pelo ffmpeg. Falsa proteção.	Removido; proteção real: int8 + threads + chunks. A thread-env é mantida.	CORRIGIDO
H2	Chamada NVIDIA sem timeout + retries internos do SDK (core/scoring.py)	Default 600 s × max_retries=2 internos × 3 nossos = stall de horas por lote num job de 1h40; contagem dupla.	Client com timeout=180 e max_retries=0; retry único em _call_with_retry.	CORRIGIDO
H3	Ctrl+C não gerava relatório (clipper.py)	Interrupção perdia a fotografia da execução; §15 exige status interrupted.	Handler BaseException → finish('interrupted') + resumo + re-raise.	CORRIGIDO (+testado)
H4	Saída ffmpeg (-y) poderia coincidir com a entrada (clipper.py)	Probabilidade baixa, impacto catastrófico (original de 17 GB sobrescrito).	Guarda out.resolve()==vide o.resolve() → clipe ignorado com erro.	CORRIGIDO

MEDIUM

ID	Achado (arquivo)	Risco / Impacto	Solução	Status
M1	faster-whisper decodifica áudio inteiro (~0,4 GB/100 min)	Pico somado ainda em poucos GB; aceitável em 16 GB.	Documentado; alternativa em chunks já existe.	ACEITO
M2	whisper.cpp recarrega o modelo ggml a cada chunk (~200× p/ 1h40)	Lentidão, sem risco de recurso (cada proc termina).	Documentado.	ACEITO
M3	Sem checagem de disco no preflight	Avançar até 0 bytes livres.	Hardening: falha < 1 GB, aviso < 5 GB.	CORRIGIDO
M4	OpenVINO export=True + download HF no 1º uso	Primeira execução lenta; GBs em ~/.cache.	Documentado.	ACEITO
M5	Word timestamps uniformes (_even_words) nos backends GPU	Snap/legenda menos precisos nesses backends.	Documentado.	ACEITO
M6	Modelo padrão EOL (410 Gone)	Scoring falha com default.	README + §8; trocar --model.	DOCUMENTADO

LOW

- L1: OMP/MKL/OPENBLAS via setdefault após início do interpretador — eficaz p/ libs importadas depois (faster-whisper é lazy); documentado.
- L2: filtros de vídeo e Haar na CPU por desenho (leves perto de transcrição/encode); sem promessa de speedup não medida.
- L3: shutil.which('nvidia-smi') por amostra do sampler — custo desprezível; ausente aqui → null honesto.

17. Limitações e conclusão

- faster-whisper é CUDA-only: sem runtime Intel, transcrição de 1h40 roda na CPU (RTF medido ~0,6–0,9 → dezenas de minutos; seguro, só lento). Com driver Level Zero/OpenCL a B580 assume via OpenVINO/Vulkan.
- Scoring depende do free tier NVIDIA (rate limit, modelos que expiram) — falhas são contidas por lote, nunca abortam o job.
- Detecção de rosto Haar é simples (webcam fixa); fallback para centro. Jogada visual silenciosa continua difícil sem visão computacional.
- Guarda saída==entrada protege o original; manifest.json e clipes usam -y apenas sobre saídas próprias

SAFE TO BEGIN VIDEO TEST — começar pelo menor vídeo real, conferir metrics/*.json (RTF, RAM peak, backend, nvidia_api) e só então avançar aos maiores.

Nada neste relatório exigiu mudar BIOS, kernel, drivers, serviços ou clocks — nenhuma alteração de sistema foi feita; apenas código do projeto + um pip? Não: nenhuma dependência nova (reportlab já estava instalado).